

ONEDRAFT

CAD-AI — Aria Powered

Computational Project Model & Geometry Engine

Version 0.1

Status: Implementation Specification

Database: PostgreSQL

API: REST / JSON

API Version: /api/v1

Identifiers: UUID

Primary Model: Versioned Computational Project Model

AI Layer: Aria Powered

Geometry Layer: Parametric / Constraint-Aware

Coordinate System: Project-local Cartesian coordinates with explicit units

Primary Output: Validated computational building model

1. ENGINEERING OBJECTIVE

The Computational Project Model is the authoritative geometric representation of the building being designed.

OneDraft shall not treat drawings as the primary representation of architectural information.

The hierarchy is:

```
USER SPECIFICATION
  ↓
PROJECT MODEL
  ↓
PARAMETERS
  ↓
CONSTRAINTS
  ↓
GEOMETRY
  ↓
VIEWS
  ↓
DRAWINGS
  ↓
DOCUMENT PACKAGE
```

Aria operates on the Project Model.

The Geometry Engine resolves the spatial consequences of model changes.

The Documentation Engine derives architectural drawings from the resulting model.

2. COMPUTATIONAL MODEL PRINCIPLE

Every architectural object shall have:

- identity
- type
- parameters
- relationships
- constraints
- geometry
- provenance
- version
- status

An object is therefore more than its graphical representation.

For example, a wall is not merely a line.

A wall contains:

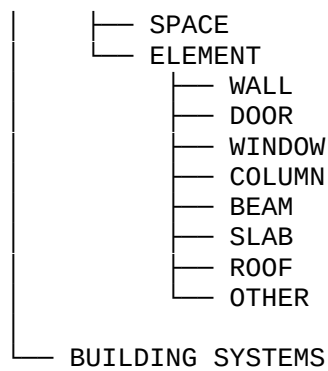
- wall identity
- host level
- location
- length
- height
- thickness
- orientation
- assembly
- openings
- connections
- constraints
- fire properties
- acoustic properties
- thermal properties
- geometry

The drawing representation is derived from these properties.

3. MODEL HIERARCHY

The initial computational hierarchy shall be:

```
PROJECT
├── SITE
│   ├── BUILDING
│   │   ├── GRID
│   │   └── LEVEL
```



The hierarchy must remain extensible.

4. MODEL OBJECT IDENTITY

Every computational object receives a UUID.

Example:

`element_id`

The UUID remains stable across ordinary geometric modifications.

Changing a wall's length does not create a new wall identity.

A new identity is created only when the semantic object itself is replaced or split according to defined model rules.

5. MODEL OBJECT STATE

Each object shall maintain:

ACTIVE
PROPOSED
SUPPRESSED
INVALID
DELETED

The state is distinct from geometric validity.

For example:

ACTIVE + VALID
ACTIVE + INVALID
PROPOSED + VALID
PROPOSED + INVALID

This distinction allows Aria to reason about proposed design changes without immediately committing them to the active model.

6. MODEL VERSION

Every coherent project state is represented by a Model Version.

Example:

```
Model Version 1
Model Version 2
Model Version 3
```

A model version identifies the complete semantic state against which geometry and documentation are evaluated.

The model version must be immutable once finalized.

7. REVISION

A Revision represents a meaningful project change.

Example:

```
Revision 1
  Initial project

Revision 2
  Revised rear wall

Revision 3
  Revised window configuration
```

A revision references a model version.

Therefore:

```
Revision
  ↓
Model Version
  ↓
Project Objects
```

8. COORDINATE SYSTEM

The geometry engine shall use a project-local Cartesian coordinate system.

Initial convention:

X = East / West
Y = North / South
Z = Vertical

The project coordinate origin shall be explicitly stored.

Example:

Origin = (0,0,0)

The project shall also maintain:

north_angle
elevation_reference
unit_system
coordinate_system

9. UNITS

All internal geometric calculations shall use a single canonical unit.

The initial canonical unit shall be:

millimetres

External units may include:

mm
cm
m
in
ft

Conversion shall occur at the API boundary or presentation layer.

The geometry engine shall not perform mixed-unit calculations.

10. VECTOR MODEL

The geometry kernel shall provide:

Vector2
Vector3

Operations:

add
subtract

multiply
divide
dot
cross
length
normalize
distance
angle
project
rotate

Example:

```
Vector3(  
    x,  
    y,  
    z  
)
```

11. POINT MODEL

A point represents a position in project coordinates.

Point2
Point3

A point shall not be treated interchangeably with a vector.

The distinction must be enforced within the geometry API.

12. TRANSFORMS

The geometry engine shall support:

translation
rotation
reflection
scaling
coordinate transformation

Architectural model elements shall normally use:

translation
rotation

Scaling of physical building elements shall not occur through arbitrary graphical transforms.

Dimensional changes must modify semantic parameters.

13. TOLERANCE MODEL

Geometric equality shall use an explicit tolerance.

Initial default:

GEOMETRIC_TOLERANCE = 0.01 mm

The tolerance shall be configurable by subsystem where necessary.

Examples:

point equality
line intersection
parallelism
coplanarity
coincidence

Exact floating-point equality shall not be used for geometric comparison.

14. GEOMETRIC PRIMITIVES

The initial geometry subsystem shall support:

Point
Vector
Line
Ray
Segment
Polyline
Arc
Circle
Plane
Polygon
Rectangle
Box
Solid

Each primitive shall have deterministic serialization.

15. LINE MODEL

A line shall be represented by:

origin
direction

A finite segment shall be represented by:

start
end

Required operations:

length
midpoint
direction
intersection
offset
trim
extend
split

16. POLYLINE MODEL

A polyline consists of ordered geometric segments.

Example:

P1 → P2 → P3 → P4

Operations:

length
offset
reverse
split
join
simplify
intersect

Polylines shall preserve vertex order.

17. POLYGON MODEL

A polygon shall contain:

outer_boundary
inner_boundaries

This permits representation of:

rooms
slabs
floor plates
openings
site boundaries

Polygon validation shall detect:

self-intersection
duplicate vertices
zero-length edges
invalid winding
open boundaries

18. PLANE MODEL

A plane shall contain:

origin
normal

Required operations:

distance_to_point
project_point
intersect_line
intersect_plane

Planes are fundamental to:

levels
walls
sections
roof geometry
drawing generation

19. SOLID MODEL

The initial solid subsystem shall support:

extrusion
box
prism
boolean_union
boolean_difference
boolean_intersection

Solids shall retain references to their originating semantic objects.

20. GEOMETRY REFERENCE

The database record:

geometry.geometry_references

does not itself become the computational geometry engine.

It provides persistent provenance.

The geometry service owns computational geometry.

The relationship is:

```
DATABASE
  ↓
geometry_reference
  ↓
GEOMETRY SERVICE
  ↓
GEOMETRIC OBJECT
```

21. PARAMETRIC ELEMENT MODEL

Architectural elements shall be generated from semantic parameters.

Example wall:

WallParameters

```
length
height
thickness
base_elevation
orientation
assembly_id
location
```

The geometry engine derives:

```
wall centerline
wall faces
wall solid
wall bounding box
```

22. WALL MODEL

A wall shall contain:

```
id
level_id
host
path
height
thickness
base_offset
top_constraint
```

assembly
orientation
openings

The wall geometry is generated from these parameters.

A wall shall not require manually stored linework to remain valid.

23. WALL PATH

A wall path may initially be:

LINE
POLYLINE
ARC

The wall generator converts the path into wall geometry.

Example:

```
centerline
  ↓
offset left
  ↓
offset right
  ↓
end conditions
  ↓
closed profile
  ↓
extrusion
  ↓
wall solid
```

24. DOOR MODEL

A door shall contain:

host_wall_id
width
height
head_height
sill_height
offset
swing
handing
type

The door opening shall be derived from its parameters.

The door must remain associated with its host wall.

25. WINDOW MODEL

A window shall contain:

host_wall_id
width
height
sill_height
head_height
offset
window_type

The geometry engine creates the opening from the semantic window object.

26. OPENING MODEL

Doors and windows shall generate standardized opening objects.

An opening contains:

host_element_id
boundary
bottom_elevation
top_elevation
opening_type

The host wall geometry engine consumes the opening set.

27. COLUMN MODEL

A column shall contain:

location
profile
width
depth
height
base_level
top_level
material

The geometry engine generates its solid representation.

28. BEAM MODEL

A beam shall contain:

- start
- end
- profile
- elevation
- orientation
- assembly

The beam generator creates the corresponding solid.

Beam connections shall be represented through the relationship graph.

29. SLAB MODEL

A slab shall contain:

- boundary
- thickness
- elevation
- assembly
- openings
- slope

Its solid is generated from the boundary and thickness.

30. ROOF MODEL

The initial roof subsystem shall support:

- roof boundary
- slope
- ridge
- valley
- eave
- thickness
- assembly

Roof generation shall be parameter-driven.

31. LEVEL MODEL

A Level is a horizontal datum.

It contains:

elevation
name
sequence
floor_to_floor_height
finished_floor_elevation
ceiling_elevation

Every level has an explicit Z elevation.

32. LEVEL RELATIONSHIPS

Levels may be related through:

ABOVE
BELOW
PRIMARY
MEZZANINE
BASEMENT
ROOF

The sequence shall remain deterministic.

33. SPACE MODEL

A Space is a semantic region.

It contains:

boundary
level
name
number
type
occupancy
area
volume

Space boundaries may be derived from surrounding building elements.

34. SPACE BOUNDARY RESOLUTION

The engine shall eventually support:

wall enclosure
column enclosure

open boundary
explicit boundary

A room does not need to be manually redrawn when a host wall moves.

The space boundary should update from the computational model.

35. BUILDING ENVELOPE

The engine shall calculate:

gross building envelope
conditioned envelope
exterior surface
roof envelope
foundation boundary

Envelope geometry becomes a major input to compliance analysis.

36. OBJECT DEPENDENCY GRAPH

The model shall maintain dependency relationships.

Example:

```
LEVEL
↓
WALL
↓
DOOR
↓
OPENING
↓
SPACE
↓
DRAWING
```

If a parent object changes, dependent objects are identified.

37. DEPENDENCY PROPAGATION

A change to an object triggers dependency analysis.

Example:

```
MOVE WALL
↓
```

```
UPDATE WALL GEOMETRY
↓
UPDATE DOOR OPENINGS
↓
UPDATE WINDOW OPENINGS
↓
UPDATE SPACE BOUNDARY
↓
UPDATE FLOOR AREA
↓
UPDATE DRAWINGS
↓
UPDATE VALIDATION
```

Only affected downstream objects should be regenerated where possible.

38. DIRTY STATE

Objects shall support computational dirty-state tracking.

Initial states:

```
CLEAN
DIRTY
PROCESSING
VALID
INVALID
```

A semantic modification marks affected objects as:

```
DIRTY
```

39. GEOMETRY REGENERATION

The geometry engine shall regenerate dirty objects in dependency order.

Conceptually:

```
identify dirty objects
↓
build dependency graph
↓
topological ordering
↓
generate geometry
↓
validate geometry
↓
commit geometry version
```

40. GEOMETRY JOB

Expensive geometry operations shall execute through worker jobs.

Example:

GeometryJob

```
project_id
model_version_id
object_id
geometry_version
operation
parameters
status
```

Statuses:

```
QUEUED
RUNNING
COMPLETED
FAILED
INVALIDATED
```

41. STALE JOB PROTECTION

A geometry worker must verify:

```
project_id
model_version_id
object_id
geometry_version
```

before committing its result.

If the model version no longer matches:

INVALIDATED

The geometry result must not become current state.

42. GEOMETRY HASH

Every generated geometry artifact shall receive a deterministic hash.

Example:

```
geometry_hash
```

The hash shall be generated from canonical geometric serialization.

This permits:

- change detection
- cache validation
- provenance
- reproducibility

43. CANONICAL SERIALIZATION

Geometric objects must have deterministic serialization.

Equivalent geometry must serialize identically after normalization.

Normalization may include:

- vertex ordering
- floating-point precision
- orientation normalization
- parameter ordering

44. GEOMETRY CACHE

Repeated geometry computation should use caching.

Cache key:

- project_id
- model_version_id
- object_id
- geometry_version
- geometry_engine_version

A geometry engine version change invalidates incompatible cached geometry.

45. ENGINE VERSIONING

The geometry engine itself shall be versioned.

Example:

- geometry-engine 0.1.0
- geometry-engine 0.1.1
- geometry-engine 0.2.0

A generated geometry artifact records the engine version used to produce it.

46. MODEL KERNEL

The computational model kernel shall expose operations such as:

```
create_object()  
get_object()  
update_object()  
delete_object()  
query_objects()  
resolve_dependencies()  
validate_object()
```

The kernel does not expose unrestricted database access.

47. COMMAND BOUNDARY

Aria must never directly manipulate geometry storage.

Instead:

```
ARIA  
↓  
COMMAND  
↓  
PROJECT MODEL SERVICE  
↓  
MODEL KERNEL  
↓  
GEOMETRY ENGINE  
↓  
VALIDATION  
↓  
REVISION
```

This is the central safety boundary of the computational model.

48. COMMAND EXECUTION

A command shall contain:

```
command_id  
project_id  
revision_id  
operation  
targets
```

parameters
constraints
requested_by

The command processor resolves the request into deterministic model operations.

49. COMMAND VALIDATION

Before execution:

verify authorization
verify project
verify revision
verify target objects
verify parameters
verify constraints
verify object state
verify model version

If validation fails, no model mutation occurs.

50. COMMAND PLANNING

Commands shall first produce an execution plan.

Example:

MOVE WALL

becomes:

1. Identify wall
2. Resolve wall geometry
3. Identify hosted openings
4. Identify affected spaces
5. Identify affected drawings
6. Identify affected validation rules
7. Calculate proposed geometry
8. Check constraints
9. Produce mutation plan

The plan may then be approved or executed according to command policy.

51. PROPOSED MODEL STATE

Aria may generate a proposed model state before committing.

Conceptually:

```
CURRENT MODEL
  ↓
PROPOSED MODEL
  ↓
VALIDATION
  ↓
ACCEPT / REJECT
  ↓
NEW REVISION
```

This permits safe design iteration.

52. CONSTRAINT ENGINE INTERFACE

The geometry engine shall expose constraints to the validation layer.

Examples:

```
distance
parallel
perpendicular
aligned
coincident
inside
outside
minimum_clearance
maximum_distance
fixed_dimension
```

Constraints are evaluated against the model.

53. DIMENSIONAL CONSTRAINT

Example:

```
Wall A
  ↑
3000 mm
  ↑
Wall B
```

The constraint engine stores:

```
distance = 3000
unit = mm
tolerance = ...
```

Moving either wall causes the constraint to be reevaluated.

54. ALIGNMENT CONSTRAINT

Objects may maintain:

- horizontal alignment
- vertical alignment
- grid alignment
- centerline alignment
- face alignment

Alignment constraints shall reference semantic objects rather than drawing coordinates.

55. GRID CONSTRAINTS

Building grids shall support:

- grid line
- grid intersection
- grid spacing
- grid labels

Elements may reference grid positions.

Example:

Column C1
↓
Grid A / Grid 3

56. HOST RELATIONSHIPS

Hosted objects must maintain host references.

Examples:

- Door → Wall
- Window → Wall
- Opening → Wall
- Fixture → Space
- Ceiling → Space

A host change triggers dependent recalculation.

57. GEOMETRIC VALIDATION

Every generated object shall pass baseline geometry checks.

Checks include:

- valid geometry
- non-zero dimensions
- closed boundaries
- non-self-intersection
- valid solid
- valid host
- valid level
- valid coordinates

58. MODEL VALIDATION

Model validation shall operate above raw geometry.

Examples:

- door hosted by wall
- window within wall boundary
- space associated with level
- element does not violate host constraints
- level elevations are coherent
- building elements occupy valid levels

59. SPATIAL QUERIES

The geometry engine shall eventually support:

- point_inside
- intersects
- contains
- touches
- overlaps
- distance
- nearest
- within
- crosses

These queries are required by:

- compliance
- coordination
- space calculation
- drawing generation
- collision detection

60. BOUNDING BOXES

Every geometry artifact shall expose:

```
min_x  
min_y  
min_z  
max_x  
max_y  
max_z
```

Bounding boxes enable fast spatial filtering before expensive geometric operations.

61. SPATIAL INDEXING

The computational geometry service shall support spatial indexing.

Initial strategy:

AABB

Advanced implementations may introduce:

```
R-tree  
BVH  
octree
```

The initial implementation should remain simple and deterministic.

62. COLLISION DETECTION

The engine shall provide a collision query between objects.

Example:

```
beam ↔ duct  
wall ↔ structural column  
door ↔ wall  
equipment ↔ clearance zone
```

Collision results are returned to the validation subsystem.

63. CLEARANCE MODEL

Clearances shall be represented as explicit geometric constraints.

Example:

Object
+
Clearance Envelope

The system evaluates whether another object intersects the envelope.

64. AREA CALCULATION

The model engine shall calculate:

gross area
net area
usable area
room area
floor area
building area

Area calculations must reference the active model version.

65. VOLUME CALCULATION

The model engine shall calculate volume for valid spaces and solids.

Example:

space volume
building volume
enclosed volume

66. SURFACE CALCULATION

The geometry engine shall expose:

surface area
exterior surface area
interior surface area
roof area
wall area

These values can later feed:

energy analysis
material quantities
cost estimation
code analysis

67. MODEL QUERY LANGUAGE

The Project Model Service shall provide structured queries.

Example:

get all walls on Level 1
get all doors hosted by Wall X
get all spaces with occupancy RESIDENTIAL
get all elements intersecting region X
get all objects affected by Element Y

Queries shall not require direct SQL from Aria or clients.

68. MODEL SNAPSHOT

A model snapshot contains the complete semantic state required to reconstruct a model version.

Conceptually:

Project
Site
Buildings
Levels
Spaces
Elements
Assemblies
Constraints
Relationships

The snapshot references immutable geometry versions.

69. MODEL RECONSTRUCTION

Given:

project_id
model_version_id

the system shall reconstruct:

semantic model
+
relationships
+
constraints
+
geometry references

without depending on a later model version.

70. MODEL DIFFERENCE

The engine shall support comparison between model versions.

Example:

Model Version 12
↓
Model Version 13

Output:

ADDED
REMOVED
MODIFIED
UNCHANGED

71. GEOMETRIC DIFFERENCE

Modified objects shall expose geometric differences.

Examples:

position changed
length changed
height changed
boundary changed
orientation changed
host changed

This information becomes an input to drawing regeneration.

72. IMPACT ANALYSIS

Before committing a command, the system shall calculate affected objects.

Example:

```
MOVE WALL
↓
wall
↓
doors
↓
windows
↓
spaces
↓
dimensions
↓
views
↓
drawings
↓
validation
```

This becomes the model's impact graph.

73. DRAWING IMPACT

The Project Model shall identify drawings affected by a model change.

Example:

```
Wall changed
↓
A101 Floor Plan
A201 Elevation
A301 Section
A401 Wall Details
```

Only affected drawings need regeneration.

74. DOCUMENT IMPACT

If a model change affects a finalized document package, the package becomes:

SUPERSEDED

A new package must be generated from the new revision.

75. COMPLIANCE IMPACT

Model changes shall identify potentially affected compliance rules.

Example:

```
door width change
↓
accessibility rules
↓
egress rules
↓
occupancy analysis
```

The validation subsystem determines the final result.

76. GEOMETRY → DOCUMENTATION INTERFACE

The geometry engine shall not generate final architectural sheets directly.

Instead it exposes:

```
model geometry
semantic objects
view geometry
annotations
dimensions
relationships
```

to the Documentation Engine.

77. VIEW GENERATION

A view is generated from:

```
model_version
view_type
camera/orientation
section plane
visibility rules
scale
```

Initial view types:

```
PLAN
ELEVATION
SECTION
```

78. PLAN VIEW

A plan view is generated by intersecting the model with a horizontal viewing configuration.

The engine identifies:

- cut geometry
- projection geometry
- overhead geometry
- annotations
- dimensions

79. SECTION VIEW

A section view contains:

- section plane
- cut geometry
- projection geometry
- level datums
- elements
- annotations
- dimensions

The section plane is a semantic model object.

80. ELEVATION VIEW

An elevation view is generated from an orientation vector.

The engine resolves:

- visible exterior elements
- projected geometry
- levels
- openings
- roof geometry
- annotations

81. DETAIL VIEW

A detail view references:

- parent view
- detail boundary
- scale
- target objects

It may include higher-resolution geometry.

82. VIEWPORT DATA

Each generated view shall produce a deterministic representation containing:

- geometry
- object_ids
- layers
- visibility
- cut_status
- annotation anchors
- dimension anchors

The Documentation Engine consumes this representation.

83. GRAPHIC SEMANTICS

Objects shall expose drawing semantics.

Example:

```
WALL
  cut_lineweight
  projection_lineweight
  hatch
  fill

DOOR
  swing_symbol
  frame_linework

WINDOW
  frame_linework
  glazing_linework
```

Graphic standards remain separate from computational geometry.

84. CAD EXPORT INTERFACE

The model shall eventually support export through a dedicated adapter.

Potential targets:

DXF
DWG
IFC
SVG
PDF

The computational model remains independent of any single export format.

85. IFC INTEROPERABILITY

IFC interoperability shall be treated as an integration boundary.

The internal OneDraft model must not be constrained to IFC's internal representation.

Mapping occurs:

```
OneDraft Model
  ↓
IFC Adapter
  ↓
IFC Model
```

86. GEOMETRY ENGINE API

Internal service interface:

```
create_geometry()
update_geometry()
delete_geometry()
regenerate_geometry()
validate_geometry()
intersect()
distance()
contains()
collision()
section()
project()
```

87. PROJECT MODEL SERVICE API

Internal service interface:

```
create_project()
create_building()
create_level()
create_space()
create_element()
update_element()
delete_element()
query_model()
create_constraint()
execute_command()
create_revision()
reconstruct_version()
compare_versions()
```

88. TRANSACTION MODEL

A semantic model mutation shall occur through:

```
BEGIN
↓
load current revision
↓
validate command
↓
construct proposed state
↓
resolve dependencies
↓
calculate geometry
↓
validate geometry
↓
validate constraints
↓
persist changes
↓
create revision
↓
record events
COMMIT
```

No partially committed semantic model is permitted.

89. ASYNCHRONOUS GEOMETRY

Where geometry computation exceeds the synchronous execution threshold:

COMMAND
↓
PROPOSED STATE
↓
GEOMETRY JOB
↓
VALIDATION
↓
FINALIZATION

The client receives:

job_id

and may monitor progress.

90. SYNCHRONOUS GEOMETRY

Simple operations may execute synchronously.

Examples:

move point
change wall length
change door width
change room name
change level elevation

The exact threshold shall be established through profiling.

91. FAILURE MODEL

Geometry failures shall not corrupt the model.

Possible states:

COMMAND_FAILED
GEOMETRY_FAILED
VALIDATION_FAILED
CONSTRAINT_CONFLICT
STALE_MODEL
STALE_GEOMETRY
INVALID_OBJECT

Each failure shall include:

error_code
message
affected_objects
resolution

request_id

92. DETERMINISM

Given identical:

model version
command
parameters
geometry engine version

the geometry engine should produce the same result.

Determinism is required for:

testing
provenance
reconstruction
debugging
document regeneration

93. IDEMPOTENT GEOMETRY

Regenerating geometry for an unchanged model version must not create arbitrary differences.

The output hash should remain stable.

94. TEST FIXTURES

The geometry subsystem shall include fixtures for:

single wall
two intersecting walls
wall with door
wall with window
four-wall room
two-level building
simple roof
simple slab
simple column
simple beam

95. UNIT TESTS

Initial tests shall verify:

vector mathematics
point/vector distinction
line intersection
polygon validity
offset geometry
wall generation
opening generation
solid generation
area calculation
volume calculation
collision detection
constraint evaluation

96. INTEGRATION TEST

The first geometry integration test shall execute:

CREATE PROJECT
↓
CREATE BUILDING
↓
CREATE GROUND LEVEL
↓
CREATE FOUR WALLS
↓
CREATE DOOR
↓
CREATE WINDOWS
↓
CREATE ROOM
↓
GENERATE GEOMETRY
↓
VALIDATE MODEL
↓
CREATE REVISION
↓
MOVE WALL
↓
REGENERATE DEPENDENCIES
↓
VERIFY ROOM
↓
VERIFY DOOR
↓
VERIFY WINDOWS
↓
VERIFY DRAWING IMPACT

97. ARIA INTEGRATION TEST

The first Aria computational test shall execute:

USER:

"Move the rear bedroom wall 300 mm north."

↓

ARIA

↓

COMMAND PLAN

↓

IDENTIFY WALL

↓

IDENTIFY DEPENDENCIES

↓

GENERATE PROPOSED STATE

↓

CALCULATE GEOMETRY

↓

CHECK CONSTRAINTS

↓

CHECK DEPENDENCIES

↓

CREATE REVISION

↓

REGENERATE AFFECTED MODEL

↓

RETURN RESULT

The response shall identify:

command_id
revision_id
model_version_id
affected_objects
affected_drawings
validation_status

98. GEOMETRY WORKER

The worker application shall eventually contain:

```
apps/worker/  
  geometry/  
    kernel/  
    primitives/  
    operations/  
    generators/  
    validators/  
    serializers/  
    cache/
```

The exact implementation language may remain Python for the initial engine.

Performance-sensitive geometry operations may later be isolated behind a compiled geometry service without changing the domain contract.

99. PYTHON PACKAGE STRUCTURE

The initial implementation should establish:

```
packages/  
  domain/  
    geometry/  
    model/  
    constraints/  
    commands/  
    revisions/
```

Example:

```
packages/domain/geometry/  
  __init__.py  
  vectors.py  
  points.py  
  lines.py  
  polygons.py  
  planes.py  
  solids.py  
  transforms.py  
  tolerance.py  
  serialization.py
```

100. PROJECT MODEL PACKAGE

The model package shall contain:

```
packages/domain/model/  
  __init__.py  
  project.py  
  site.py  
  building.py
```

```
level.py
space.py
element.py
wall.py
door.py
window.py
slab.py
roof.py
beam.py
column.py
assembly.py
relationship.py
constraint.py
```

101. COMMAND PACKAGE

The command subsystem shall contain:

```
packages/domain/commands/
  command.py
  planner.py
  executor.py
  validator.py
  impact.py
  results.py
```

102. GEOMETRY ENGINE PACKAGE

The worker-side geometry engine shall contain:

```
apps/worker/geometry/
  engine.py
  dispatcher.py
  jobs.py
  cache.py
  generators/
  validators/
```

103. MODEL SERVICE PACKAGE

The API-side Project Model service shall contain:

```
apps/api/
  services/
    project_model_service.py
    command_service.py
    revision_service.py
```

104. REPOSITORY BOUNDARY

Domain objects must not depend directly on PostgreSQL implementation details.

The architecture shall use repository interfaces.

Example:

```
ProjectRepository
ModelRepository
ElementRepository
RevisionRepository
ConstraintRepository
GeometryRepository
```

Infrastructure implements those interfaces.

105. DOMAIN / INFRASTRUCTURE SEPARATION

The dependency direction shall be:

```
API
↓
APPLICATION SERVICES
↓
DOMAIN
↑
INFRASTRUCTURE ADAPTERS
```

The domain shall not depend on:

```
FastAPI
PostgreSQL
SQLAlchemy
Redis
Docker
AI provider
```

106. PERSISTENCE ADAPTER

The PostgreSQL implementation shall translate:

Domain Object
↓
Persistence Model
↓
Database Row

The database schema remains the persistence contract already established in OneDraft v0.1.

107. EVENT EMISSION

A successful command produces domain events.

Examples:

ProjectCreated
ElementCreated
ElementModified
GeometryGenerated
ConstraintViolated
RevisionCreated
DrawingInvalidated

Events are written to:

`audit.project_events`

and may later feed asynchronous services.

108. EVENT ORDER

Events shall preserve causal order.

Example:

ElementModified
↓
GeometryRegenerated
↓
ValidationExecuted
↓
RevisionCreated
↓
DrawingInvalidated

Each event references the applicable:

project_id
revision_id
object_id

109. OBSERVABILITY

Every computational operation shall expose:

```
request_id  
command_id  
project_id  
revision_id  
model_version_id  
job_id  
duration  
status
```

This allows complete tracing across:

```
API  
↓  
command service  
↓  
model kernel  
↓  
geometry worker  
↓  
validation  
↓  
documentation
```

110. PERFORMANCE TARGETS

Initial engineering targets:

Simple model query:

< 100 ms

Simple semantic modification:

< 500 ms

Simple geometry regeneration:

< 2 seconds

Complex geometry and document generation shall be asynchronous.

These are engineering targets rather than contractual guarantees and shall be revised after profiling.

111. CONCURRENCY CONTROL

Project mutations shall use optimistic concurrency.

Every mutation references:

`expected_revision`

If the expected revision is not current:

`STALE_MODEL`

The command is rejected rather than silently rebased.

112. MODEL LOCKING

Long-running geometry operations shall not hold database locks for their entire duration.

Instead:

```
snapshot
↓
compute
↓
verify version
↓
commit
```

This prevents expensive geometry operations from blocking unrelated project reads.

113. CACHE INVALIDATION

Geometry cache entries become invalid when any of the following changes:

```
model_version
object parameters
dependencies
geometry engine version
assembly
coordinate system
unit system
```

114. ASSEMBLY DEPENDENCY

An element references an assembly.

Changing the assembly may affect:

- thickness
- fire rating
- thermal properties
- acoustic properties
- material layers
- geometry

The dependency engine therefore treats assemblies as model dependencies.

115. MATERIAL LAYER GEOMETRY

Assemblies may contain:

- finish
- membrane
- insulation
- sheathing
- stud
- air_space
- structure

The initial geometry engine may represent the complete assembly as a single solid.

Layer-by-layer geometry becomes an advanced capability.

116. MODEL SEMANTICS

Geometry alone is insufficient.

The model must distinguish:

- WALL
- DOOR
- WINDOW
- SLAB
- ROOF
- COLUMN
- BEAM
- SPACE
- LEVEL
- GRID

Two geometrically identical objects may have different semantic meaning.

117. SEMANTIC VALIDITY

The engine shall validate both:

GEOMETRIC VALIDITY

and:

SEMANTIC VALIDITY

Example:

A rectangular solid may be geometrically valid but invalid as a door because it lacks a valid host wall.

118. MODEL INVARIANTS

Initial invariants include:

Every element belongs to a project.

Every level belongs to a building.

Every space belongs to a level.

Every hosted opening references a valid host.

Every geometry reference belongs to a model object.

Every revision references a model version.

Every generated drawing references a model version.

Every finalized document references a revision.

119. MODEL HEALTH

The Project Model Service shall expose a health assessment.

Example:

MODEL HEALTH

Geometry: VALID

Relationships: VALID

Constraints: 2 WARNINGS

Spaces: VALID

Assemblies: VALID

Documentation dependencies: 3 DIRTY

This becomes an internal diagnostic capability for Aria.

120. ARIA MODEL CONTEXT

When Aria receives a project command, the orchestration layer shall provide a structured model context.

Example:

```
project
current_revision
current_model_version
selected_objects
related_objects
constraints
requirements
affected_drawings
recent_changes
```

Aria should not need unrestricted database access to understand the model.

121. ARIA ACTION SPACE

Aria's executable actions shall be limited to registered domain commands.

Examples:

```
CREATE_ELEMENT
MOVE_ELEMENT
ROTATE_ELEMENT
RESIZE_ELEMENT
DELETE_ELEMENT
CHANGE_PARAMETER
CREATE_CONSTRAINT
REMOVE_CONSTRAINT
CREATE_VIEW
GENERATE_DRAWING
RUN_VALIDATION
```

122. UNKNOWN COMMAND HANDLING

Natural-language requests that cannot be mapped safely to a registered operation shall not modify the model.

The system returns:

UNRESOLVED_COMMAND

with the unresolved intent and available context.

123. COMMAND EXPLANATION

Each executed command shall retain:

original_request
normalized_operation
target_objects
parameters
constraints
result

This provides traceability between natural language and computational mutation.

124. PROVENANCE

Every model mutation shall be attributable to:

USER
ARIA
SYSTEM
IMPORT
INTEGRATION

The source is stored in the revision/change record.

125. IMPORT MODEL

Imported geometry shall enter the system through an import boundary.

Example:

IFC
DXF
DWG

becomes:

IMPORT
↓
NORMALIZATION
↓
SEMANTIC MAPPING
↓

VALIDATION

↓

PROJECT MODEL

Imported geometry must not bypass validation.

126. EXTERNAL MODEL REFERENCES

External references shall maintain:

source
source_version
source_hash
import_timestamp
mapping_version

This allows imported model provenance to be reconstructed.

127. COMPUTATIONAL PROJECT MODEL CONTRACT

The Project Model Engine guarantees:

stable object identity
versioned state
parameterized geometry
constraint awareness
dependency tracking
deterministic regeneration
model reconstruction
impact analysis
drawing dependency tracking

128. FIRST EXECUTABLE ARTIFACTS

The next implementation package shall produce:

packages/domain/geometry/vectors.py
packages/domain/geometry/points.py
packages/domain/geometry/lines.py
packages/domain/geometry/polygons.py
packages/domain/geometry/transforms.py
packages/domain/model/element.py
packages/domain/model/wall.py
packages/domain/model/door.py


```
packages/domain/model/window.py
packages/domain/model/level.py
packages/domain/model/space.py
packages/domain/commands/command.py
packages/domain/commands/planner.py
packages/domain/commands/executor.py
apps/api/services/project_model_service.py
apps/worker/geometry/engine.py
```

129. FIRST IMPLEMENTATION SEQUENCE

The implementation order shall be:

1. Geometry primitives
 2. Geometry tolerance
 3. Vector mathematics
 4. Transform system
 5. Parametric elements
 6. Level model
 7. Wall model
 8. Opening model
 9. Door model
 10. Window model
 11. Space model
 12. Dependency graph
 13. Constraint interface
 14. Model kernel
 15. Command planner
 16. Command executor
 17. Geometry worker
 18. Revision integration
 19. Validation integration
 20. Drawing impact integration
-

130. FIRST MODEL TEST

The first executable building model shall be:

PROJECT
↓
BUILDING
↓
GROUND LEVEL
↓
FOUR EXTERIOR WALLS
↓
ONE INTERIOR WALL
↓
ONE DOOR
↓
TWO WINDOWS
↓
ONE ROOM

The system must generate valid computational geometry from semantic parameters.

131. FIRST MODIFICATION TEST

The system shall then execute:

MOVE INTERIOR WALL
300 mm NORTH

The expected propagation is:

WALL
↓
DOOR
↓
ROOM BOUNDARY
↓
ROOM AREA
↓
GEOMETRY
↓
AFFECTED VIEWS
↓
AFFECTED DRAWINGS

132. FIRST REVISION TEST

The modification must produce:

Revision 2

Model Version 2

while preserving:

Revision 1

Model Version 1

The original geometry must remain reconstructable.

133. FIRST RECONSTRUCTION TEST

The system shall load:

Model Version 1

and verify that it produces the original building state.

It shall then load:

Model Version 2

and verify the modified state.

No later state may contaminate either reconstruction.

134. FIRST GEOMETRY HASH TEST

For an unchanged model version:

```
generate geometry
hash geometry
generate geometry again
hash geometry
```

The hashes must match.

135. FIRST STALE JOB TEST

The system shall execute:

Model Version 1

↓

start geometry job

↓

create Model Version 2

↓

complete geometry job for Version 1

The result must be:

INVALIDATED

and must not modify current Version 2 state.

136. FIRST CONSTRAINT TEST

Create:

Wall A

Wall B

Distance Constraint = 3000 mm

Move Wall B to:

3500 mm

The system must identify:

CONSTRAINT_CONFLICT

without silently accepting the invalid state as compliant.

137. FIRST ARIA TEST

Aria receives:

Move the rear bedroom wall 300 mm north while keeping the exterior envelope unchanged.

The system must transform the request into:

operation = MOVE_ELEMENT

target = rear bedroom wall

distance = 300 mm

direction = NORTH

preserve_constraint = exterior envelope

The command is then evaluated through the Project Model Service.

138. GEOMETRY ENGINE SUCCESS CRITERIA

The Geometry Engine is successful when it can:

Represent architectural geometry

Generate parametric building elements

Maintain object identity

Maintain host relationships

Resolve dependencies

Evaluate geometric constraints

Regenerate affected geometry

Detect stale computations

Produce deterministic geometry

Calculate areas and volumes

Perform spatial queries

Support plan/section/elevation extraction

Maintain geometry provenance

Reconstruct historical model versions

139. PROJECT MODEL SUCCESS CRITERIA

The Project Model is successful when:

Natural Language



Aria Command



Domain Operation



Model Mutation



Geometry



Validation



Revision

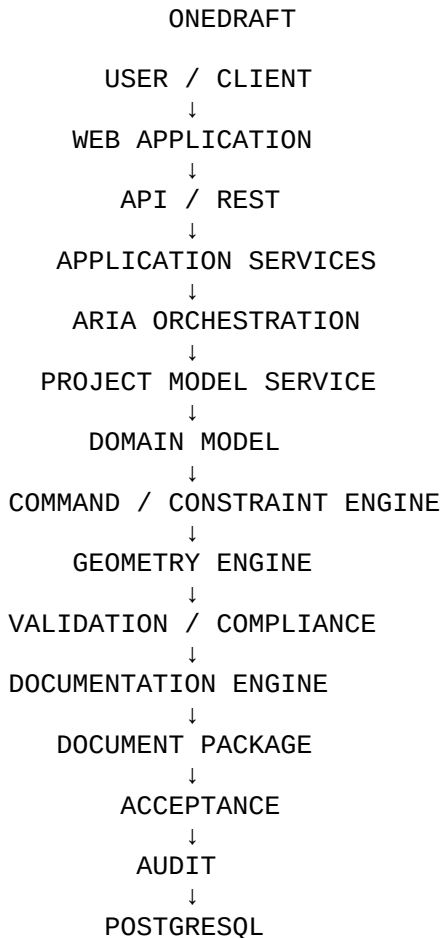


Drawing Impact

can occur without direct AI access to PostgreSQL.

140. ARCHITECTURAL STACK POSITION

The OneDraft stack now becomes:



Workers and event processing operate alongside the computational layers rather than bypassing them.

141. NEXT STACK LAYER

With the computational Project Model and Geometry Engine established, the next major stack layer is:

REGULATORY / COMPLIANCE ENGINE

Its purpose is to transform jurisdiction-specific regulatory sources into executable applicability rules and validation operations.

The sequence becomes:

PROJECT MODEL
↓
GEOMETRY
↓
REGULATORY MANIFEST
↓
RULE RESOLUTION
↓
COMPLIANCE VALIDATION
↓
VALIDATION RESULTS
↓
DOCUMENTATION

142. REGULATORY ENGINE PRECONDITION

The Regulatory Engine shall consume:

Project
Site
Jurisdiction
Occupancy
Building Type
Construction Type
Levels
Areas
Heights
Exits
Doors
Windows
Spaces
Assemblies
Geometry
Requirements

It shall not independently reconstruct the building model.

The Project Model remains authoritative.

143. REGULATORY ENGINE BOUNDARY

The architecture therefore establishes:

REGULATORY SOURCE
↓
CODE SOURCE
↓
CODE RULE
↓

```
CODE MANIFEST
↓
APPLICABILITY ENGINE
↓
VALIDATION RULE
↓
MODEL QUERY
↓
RESULT
```

The regulatory engine evaluates the model.

It does not modify the model without an explicit command.

144. FINAL ARCHITECTURAL DECISION

OneDraft's computational core is therefore defined as:

```
SEMANTIC PROJECT MODEL
+
PARAMETRIC GEOMETRY
+
DEPENDENCY GRAPH
+
CONSTRAINT ENGINE
+
VERSION CONTROL
+
ARIA COMMAND BOUNDARY
```

The drawing remains a derived representation.

The geometry remains a derived representation of semantic architectural objects.

The semantic Project Model remains the central computational authority.

145. SPECIFICATION STATUS

ONEDRAFT COMPUTATIONAL PROJECT MODEL & GEOMETRY ENGINE v0.1

STATUS: ESTABLISHED

The persistence/API contract provides the machine interface.

The Computational Project Model provides the semantic building representation.

The Geometry Engine provides deterministic spatial computation.

The next implementation stage is therefore:

REGULATORY / COMPLIANCE ENGINE

followed by:

VALIDATION ENGINE

then:

DOCUMENTATION / CAD SHEET ENGINE

then:

ARIA ORCHESTRATION

then:

CUSTOMER REVIEW / ACCEPTANCE

The OneDraft stack is consequently progressing from:

PERSISTENCE



API



DOMAIN MODEL



GEOMETRY



REGULATORY INTELLIGENCE



VALIDATION



DOCUMENTATION



ARIA ORCHESTRATION



CUSTOMER ACCEPTANCE

.